Dibris Dipartimento di Informatica, Bioingegneria, Robotica e Ingegneria dei Sistemi

Augmented and Virtual Reality - code 80459 - a.y. 2025/2026

Extending Cosys-AirSim for Multi-Domain Vehicle Simulation

Bringing Aerial and Ground Robotics Together in One Simulation

---

Advisor(s): Manuel Delucchi, Saverio Iacono, Gianni Viardo Vercelli
Student(s): Andrea Valentino Ricotti, Walter Signoretti, Azzurra Suffia

UNIVERSITY OF GENOA, APRIL 2026

Member list & Workload

No.	Full name	Student ID	Percentage of work
1	Andrea Valentino Ricotti	5306194	100%
2	Walter Signoretti	5240038	100%
3	Azzurra Suffia	5341007	100%

Abstract

We enhanced Cosys-Airsim, an Unreal Engine plugin based on Microsoft AirSim, by implementing support for diverse, multi-vehicle fleets. Previously limited to single-type setups, our modification enables concurrent interactions between diverse vehicles like UAVs and UGVs. By assigning unique server ports to each vehicle and introducing new logic classes, we facilitate complex, multi-modal simulation scenarios formerly unsupported by the platform.

Contents

1 Introduction **5**
1.1 State of Art . . . 5
1.2 Motivation . . . 6
1.3 AirLib, AirSim & Cosys-AirSim . . . 6

2 System Configuration & Hardware Requirements **8**
2.1 Hardware Requirements . . . 8
2.2 Software Prerequisites . . . 8

3 Cosys-AirSim **9**
3.1 Overview . . . 9
3.2 Installation & Configuration . . . 9
3.3 Software Design & Core Classes . . . 10
3.3.1 AirLib: The Simulation Core . . . 10
3.3.2 Unreal Engine: The Rendering and Interaction Layer . . . 11
3.3.3 The Simulation API Bridge . . . 12

4 Multidomain Extension **14**
4.1 Overview . . . 14
4.1.1 Target Audience . . . 14
4.1.2 Example of Use Case: Torre dei Conti . . . 14
4.2 Integration Steps . . . 15
4.2.1 Step 1: Create the C++ Class in Unreal Engine . . . 15
4.2.2 Step 2: File Replacement Registry . . . 16
4.2.3 Step 3: Clean and Rebuild Environment . . . 16
4.3 Implementation Details . . . 17
4.3.1 Core Class Modifications . . . 17
4.3.2 Development Environment & Toolchain . . . 20
4.4 Scope and Limitations . . . 21

5 Results **22**

6 Conclusion **24**

1 Introduction

1.1 State of Art

The field of robotics has witnessed a significant paradigm shift from single-agent missions to complex Multi-Robot Systems (MRS) and Heterogeneous Multi-Agent Systems (MAS). Research in this area, which dates back to the late 1980s [3], has demonstrated that coordinated teams of robots can achieve mission objectives that are beyond the capabilities of individual units. Heterogeneous systems, characterized by agents with diverse physical structures and capabilities, offer enhanced fault tolerance, scalability, and the ability to perform multi-faceted tasks through the fusion of multi-modal data.

The integration of Unmanned Aerial Vehicles (UAVs) and Unmanned Ground Vehicles (UGVs) is a prominent example of such synergy. While UAVs provide rapid exploration and global situational awareness from an aerial perspective, UGVs contribute with extended endurance, higher payload capacities, and stable interaction with the ground environment. This collaborative approach has found critical applications in structural monitoring of historical infrastructure, as well as in Search and Rescue (SAR) [4], industrial inspection [2] and precision agriculture tasks.

Despite its potential, UAV-UGV collaboration faces significant scientific challenges [6], particularly in optimizing communication and coordination under unstable network conditions. Recent research has explored solutions like the Unified Robotic Protocol (URP) to manage communication between diverse frameworks [9], and resource-efficient strategies like Semantic Communication (SemCom) [10], which prioritizes the transmission of task-relevant semantic information over raw data to reduce bandwidth overhead.

The complexity of these systems necessitates high-fidelity simulation environments for safe and cost-effective validation. While traditional tools like Gazebo and Webots provide robust physics-based simulation, they often lack the photorealistic rendering required for advanced vision-based perception algorithms. In this context, simulators built on high-performance game engines, such as Microsoft's AirSim [8], have become state-of-the-art by offering realistic environmental physics and sensor behavior, as highlighted in recent comparative reviews [7].

Building upon this foundation, Cosys-AirSim [5] was developed to address specific industrial and research needs, including more detailed sensor models and procedurally generated environments. However, a persistent limitation in existing frameworks, including the baseline AirSim implementation, is the lack of support for heterogeneous multi-vehicle configurations in unified scenes. The proposed project addresses this gap by extending the Cosys-AirSim architecture to support concurrent interaction and control of UAVs and UGVs. This enhancement facilitates the integrated testing of perception and coordination strategies, positioning the platform as a leading tool for multi-domain autonomous research in immersive and virtual environments.

1.2 Motivation

This project aims to extend **Cosys-AirSim**, an open-source framework built upon Microsoft's AirSim simulator, which provides high-fidelity, physics-based environments for autonomous vehicles. While the current framework offers robust tools for simulating either Unmanned Aerial Vehicles (UAVs) or Unmanned Ground Vehicles (UGVs), it is restricted to single-domain operations, forcing users to choose only one vehicle type per simulation instance. The current inability of Cosys-AirSim to support concurrent UAV and UGV simulation creates a bottleneck for researchers and industries needing to validate multi-domain systems in a unified virtual space.

Figure 1: Example view of a UAV produced by Cosys-AirSim.

Figure 2: Example view of a UGV produced by Cosys-AirSim.

By modifying the underlying architecture to support simulations involving different vehicle domains simultaneously, this project seeks to bridge the gap between simulation and real-world cross-domain deployment. Furthermore, as an extension of an open-source tool, this work contributes to the broader robotics community by providing a flexible, accessible platform for testing cooperative strategies, ultimately reducing the risks and costs associated with physical multi-robot experimentation.

1.3 AirLib, AirSim & Cosys-AirSim

To fully grasp the scope of this extension, it is necessary to understand the underlying architecture of the simulation ecosystem and the specific differences between AirLib, AirSim [8], and Cosys-AirSim [5].

- **AirLib:** This is the foundational, engine-independent C++ library that serves as the core "brain" of

the simulator. It handles the underlying physics, vehicle dynamics, and external API communications, functioning completely independently of the visual rendering engine.

- **AirSim:** Originally developed by Microsoft Research, AirSim is the complete open-source framework that encapsulates AirLib and bridges it with Unreal Engine via dedicated plugins. However, Microsoft has officially archived the open-source repository, ceasing further updates and leaving the original framework incompatible with newer technologies.
- **Cosys-AirSim:** To keep the project alive and modernize its capabilities, Cosys-Lab created Cosys-AirSim as a direct fork of the original AirSim. Beyond simply maintaining the code, Cosys-Lab significantly extended the framework by migrating it to Unreal Engine 5, enhancing the sensor suite, and improving overall fidelity. However, like its predecessor, it still has the limitation of only supporting a single vehicle domain per simulation session.

2 System Configuration & Hardware Requirements

2.1 Hardware Requirements

To ensure the high-fidelity physics and graphical rendering of the Cosys-AirSim ecosystem, the following hardware specifications are recommended for all users:

- **Processor:** x86-64 architecture (although UE5.1+ supports ARM).
- **Memory:** A minimum of 8 GB RAM is required, though 16 GB is strongly recommended.
- **Storage:** At least 100 GB of available disk space is recommended to accommodate the full ecosystem, including Unreal Engine, the simulator, and development tools.
- **Graphics:** A dedicated GPU is necessary to support high graphical settings and maintain the frame rates required for accurate real-time sensor simulation.

2.2 Software Prerequisites

The software environment is primarily optimized for Windows to ensure full compatibility with simulation and debugging tools. Requirements vary depending on whether the user intends to modify the source code:

- **Operating System:** Windows 11 is the preferred platform. While Ubuntu is supported for execution, it is not recommended for core development as it lacks support for the Visual Studio IDE.
- **Simulation Engine:** The required simulation environment consists of Unreal Engine 5.5 and the Cosys-AirSim v3.3 framework.
- **Integrated Development Environment (for Developers):** Visual Studio 2022 is the suggested IDE. Note that installing Unreal Engine and attempting to create a project will typically prompt the user to install Visual Studio Community 2022 if it is not already present.
- **Version Control (for Developers):** Git and GitHub are required for source code management and collaborative contributions.
- **Programming Languages:** While Python and C++ are both used for vehicle control and API interaction, framework modifications strictly require C++.

3 Cosys-AirSim

3.1 Overview

An Unreal Engine plugin is a modular collection of code and data that extends the engine's core functionality. In general, these plugins are installed by placing the plugin folder into the `Plugins` directory of a specific project, allowing the engine to recognize and load the additional features upon startup.

3.2 Installation & Configuration

The following step-by-step guide assumes the software and hardware prerequisites discussed in Section 2 and uses the `Blocks` environment provided by the official Cosys-AirSim documentation as the base project; however, it remains applicable to any Unreal Engine 5.5 environment.

Step 1: Install Cosys-AirSim: The plugin installation instructions are provided by the official documentation.

Step 2: Set up the Environment: For the `Blocks` environment, follow the instructions provided. Optionally, navigate to the `Cosys-AirSim/Unreal/Environments` directory and copy the `Blocks` folder to your desired working directory; this folder will serve as the primary Unreal Engine project.

Step 3: Integrate the Extended Plugin: Ensure that the `AirSim` folder is present within your project's `Plugins` directory. Otherwise, copy and paste the folder from the material downloaded in Step 1 into this directory.

Step 4: Workspace Cleanup: To ensure a clean build, delete the following temporary directories if they exist in both the root folder and the `Plugins/AirSim` folder: `Binaries`, `Intermediate`, `Saved`, and `DerivedDataCache`.

Step 5: Project Compilation: You may compile the project using one of two methods:

- **Visual Studio:** Right-click the `Blocks.uproject` file, select *Generate Visual Studio project files*, open the resulting `.sln` file, and build the solution in Visual Studio.
- **Direct Launch:** Open the `Blocks.uproject` file directly. When prompted to rebuild the `Blocks` and `AirSim` modules, click *Yes* and wait for the compilation to complete.

Step 6: Configuration of Simulation Settings: To specify custom settings (e.g., number of vehicles in the simulation), you must configure the `settings.json` file. This file must be located in the user's `Documents/AirSim` directory. Otherwise, a dialog sequence prompting the user to choose the simulation mode will appear at the start of the simulation, and default settings will be assumed for all other parameters.

Step 7: Execution: Once the Unreal Editor is open, press the **Play** button. You can then execute your control scripts via a terminal.

3.3 Software Design & Core Classes

Cosys-AirSim's architecture is driven by a robust system of specialized classes. This section examines these core components. The fundamental distinction within this architecture is the division of labor: AirLib serves as a standalone C++ library responsible for the mathematical, engine-agnostic simulation logic, while the Unreal Engine plugin manages the 3D environment rendering, collision detection, and user interface.

3.3.1 AirLib: The Simulation Core

AirLib core components include:

`AirSimSettings:` It parses the `settings.json` file, defines the configuration data structures (e.g., for vehicles, their type or initial spawn positions/orientations), manages default fallbacks when information is omitted, and operates as a globally accessible singleton. It defines ten vehicle types, acting as a pre-packaged combination of the physical body, the physics engine, and the firmware interface, and supports four simulation modes (i.e., multirotor, car, skid vehicle and computer vision).

Physics engine: This is a header-only physics engine. It is designed to be fast and extensible to implement different vehicles. It calculates forces, torques, velocities, and accelerations for the vehicles. It receives collision data from the Unreal Engine wrapper and calculates the resulting physical response (e.g., bouncing or crashing). `FastPhysicsEngine` updates the exact position and orientation of the vehicles at extremely high frequencies. It handles physics for drones; while for cars, the simulation relies entirely on Unreal Engine (`Chaos Physics` in UE5). On the other hand, `ExternalPhysicsEngine` allows the drone to be controlled via `setVehiclePose()`, keeping the drone in place until the next call. It is especially useful for moving the drone using an external simulator or on a saved path.

Sensor models: These are header-only models for **sensors**. While Unreal Engine renders the visual data for cameras, AirLib handles the mathematical simulation of non-visual sensors. It takes the "Ground Truth" data from the physics engine and applies realistic noise, drift, and physics principles to generate sensor readings. Sensors include IMUs, LiDARs, GPS and others.

Control library: This component provides an abstract base class for APIs and concrete implementations for platforms like MavLink. It enables programmatic control via Python and C++. AirLib operates an RPC server (default port 41451), while the `ApiProvider` routes incoming commands to the appropriate vehicle instance.

Clocks: AirLib utilizes custom clocks to manage simulation time. The `ScalableClock` allows the simulation to run faster or slower than real-time, while the `SteppableClock` decouples simulation time from wall-clock time, advancing only in discrete steps upon command.

Firmware: Multirotor and car **firmware** acts as the vehicle's controllers. It takes high-level commands (like "fly to 50 meters"), reads the simulated sensor data, and runs complex control loops to output the exact parameters needed to execute that command. `SimpleFlight` is AirSim's custom, built-in flight controller.

Other open-source controllers are supported, such as `ArduCopter` and `PX4` for drones and `ArduRover` for cars.

3.3.2 Unreal Engine: The Rendering and Interaction Layer

The Unreal Engine plugin transposes AirLib's logic into a 3D environment.

`ASimHUD`**:** The core class responsible for the Head-Up Display (HUD), handling the visual overlay displayed on the screen when the simulation is running. In its `BeginPlay()` method, it attempts to find and read `settings.json`. If found, it parses the file and passes the data to the initialization function; if omitted, it triggers a routine to generate a default settings file. If `SimMode` is missing or empty, an Unreal Engine UI prompt is displayed asking the user to manually click and choose whether they want to spawn a Car, a Quadrotor, or a Skid vehicle.

`ASimMode` **Hierarchy:** A structured class system defining the simulation's operational rules:

- `ASimModeBase`: The foundational root class for all simulation modes.
- `ASimModeWorldBase`: Extends the base functionality by integrating the physics engine.
- `ASimModeCar/SkidVehicle`: Specialized implementations for wheeled and differential-drive robotics.
- `ASimModeComputerVision`: A lightweight, physics-disabled mode optimized for high-speed synthetic data generation.
- `ASimModeWorldMultirotor`: A dedicated implementation for aerial robotics and drone flight dynamics.

Vehicle Pawns: The visual embodiments of vehicles (e.g., `AMultirotorPawn`).

Recording: A non-blocking pipeline to continuously capture visual data, ensuring high-fidelity dataset generation without degrading the simulation's real-time performance.

`SimJoyStick:` A class handling the manual, human-in-the-loop route to control vehicles.

The relationship and inheritance structure of these simulation modes, originating from the foundational base classes to specialized vehicle implementations, is illustrated in the UML hierarchy in Figure 3.

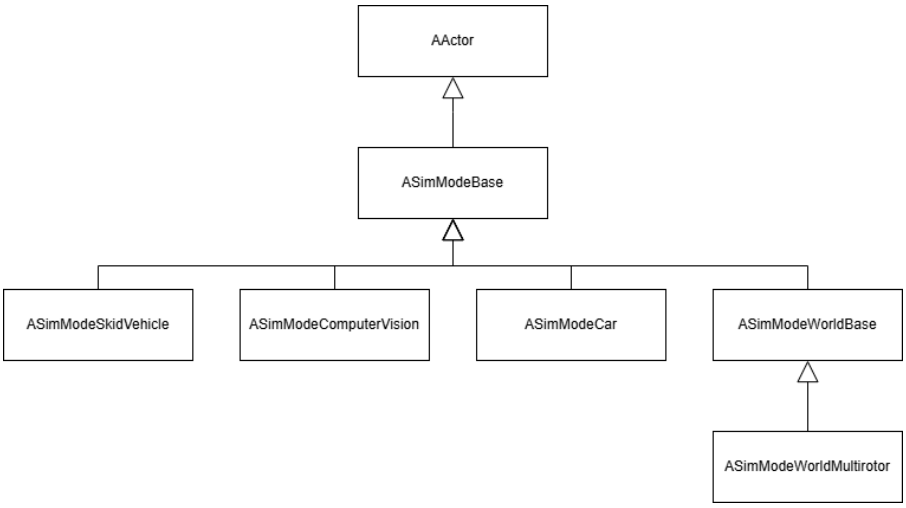

Figure 3: UML Class Hierarchy for Cosys-AirSim Simulation Modes.

The initialization process and interaction between the Unreal Engine and the AirSim plugin are illustrated in Figure 4.

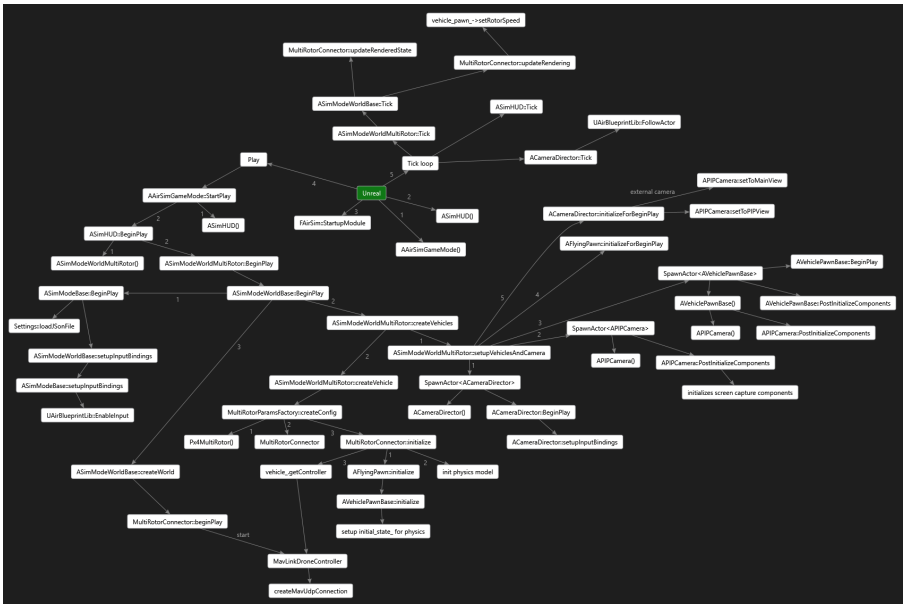

Figure 4: AirSim loading and invocation by Unreal Engine from the official documentation.

3.3.3 The Simulation API Bridge

This critical layer manages the bidirectional data flow between the core library and the rendering engine.

`PawnSimApi:` The primary mediator. It subscribes to `pawn_events` to catch Unreal collisions and frame ticks,

while simultaneously implementing the `VehicleSimApiBase` from AirLib to update the vehicle's 3D transform (position and orientation) based on physics calculations.

Vehicle-Specific APIs: Classes like `CarSimApi` and `MultirotorSimApi` extend the base functionality to handle unique actuators, such as wheel rotation, suspension compression, or rotor blur effects, ensuring the visual representation matches the internal physical state.

4 Multidomain Extension

4.1 Overview

The base architecture of Cosys-AirSim, inherited from its predecessor Microsoft AirSim, was robust yet fundamentally limited by a strict single-domain paradigm. A simulation session could be configured to host either Unmanned Aerial Vehicles (UAVs) or Unmanned Ground Vehicles (UGVs), but never both simultaneously. This represented a severe bottleneck for modern robotics research, which increasingly relies on Heterogeneous Multi-Agent Systems (MAS) to solve complex real-world problems.

To bridge this critical gap, this project develops a comprehensive multidomain extension. Hence, this extension transitions Cosys-AirSim from a simple vehicle emulator into a holistic testing ground for advanced multi-agent coordination. At its core, the extension introduces a novel architectural component: the `SimModeWorldMultidomain` class. This specialized C++ module acts as an orchestrator within the Unreal Engine environment, capable of dynamically allocating dedicated communication ports and handling disparate vehicle physics models concurrently.

The impact of this development is relevant, particularly within the realm of AI-powered robotics. By enabling an environment where aerial and ground vehicles coexist seamlessly, it unlocks entirely new experimental paradigms. For instance, it becomes possible to train reinforcement learning models where a drone acts as an "eye in the sky," streaming fused sensor data to an edge-AI algorithm that actively guides a ground rover through hazardous terrain.

4.1.1 Target Audience

The versatile nature of this environment caters to a broad spectrum of stakeholders, structured according to their level of engagement with the platform's core features:

Primary Users: include industrial and civil engineers focused on infrastructure inspection, construction, and cultural heritage preservation, as well as emergency response coordinators managing search and rescue missions.

Secondary Users: encompass specialized sectors such as agricultural engineers exploring smart farming automation, as well as defense and security professionals who leverage controlled simulations to evaluate tactical coordination between autonomous aerial and ground units, and academic researchers advancing the field of cooperative robotics.

4.1.2 Example of Use Case: Torre dei Conti

In **industrial inspection and construction**, the extended Cosys-AirSim framework allows engineers to simulate the behaviour of UAVs and UGVs around fragile historical buildings, an application that could reduce the need for full human teams in potentially dangerous contexts. Given the delicate nature of such environments,

it is crucial to minimize errors that could result in cultural losses or more severe consequences, as incidents like the partial collapse of Torre dei Conti (Rome, Nov 2025) [1] demonstrate. In these contexts, drones are already valuable tools (Corriere della Sera), even though UAV–UGV configurations remain uncommon. The simulation focuses on assessing how robots move, sense, and coordinate in environments where structural stability is uncertain and even minor trajectory errors could exacerbate existing damage. By virtually testing flight paths, collision avoidance, take-off and landing procedures, and multi-robot cooperation, users can identify risky manoeuvres, communication blind spots, or areas with insufficient sensor coverage. This preparation helps refine mission logic, adjust autonomous behaviours, and design safer inspection strategies before approaching the real site, reducing overall risk.

Figure 5: Torre de' Conti in its original state prior to the November 2025 collapse.

4.2 Integration Steps

To integrate the Multidomain Extension, begin by cloning the repository:

```
git clone https://github.com/AndyVale/AVR_2026_Project.git
```

4.2.1 Step 1: Create the C++ Class in Unreal Engine

1. Launch the `.uproject` file (e.g., `Blocks.uproject`) that utilizes the Cosys-AirSim plugin.
2. Initialize the class:
   - Navigate to `Tools` → `New C++ Class...`
   - Select `All Classes`, search for `SimModeWorldBase`, and click `Next`.

3. Configure the files:

   Name: `SimModeWorldMultidomain`

   Path: `.../Plugins/AirSim/Source/Vehicles/Multidomain/`

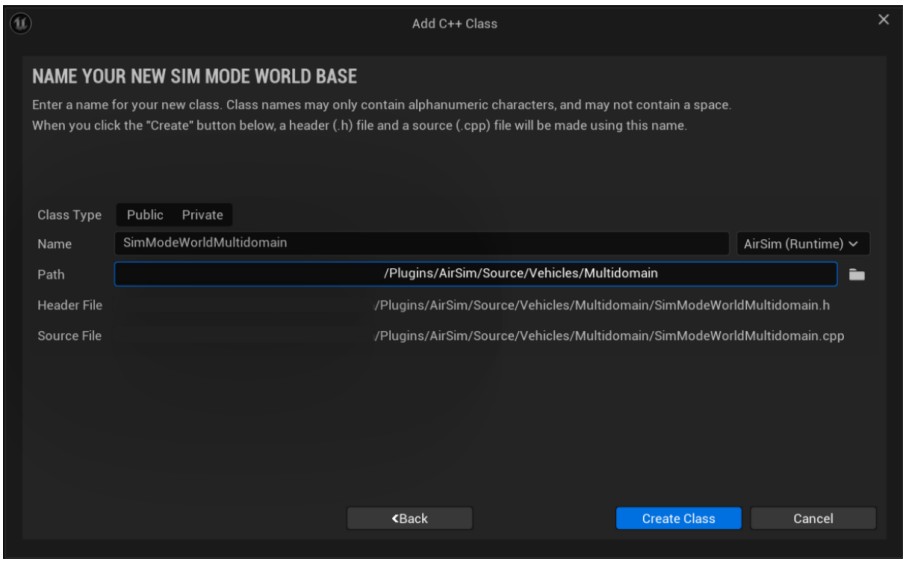

Figure 6: Unreal Engine dialog showing the class name and destination path during extension integration.

4. Click `Create Class` and wait for the *Live Coding* notification to complete.

4.2.2 Step 2: File Replacement Registry

Replace the following existing files in your local directory with the modified versions from the cloned repository:

Directory Path (.../Plugins/AirSim/Source/)	Files to Replace
`AirLib/include/common/`	`AirSimSettings.hpp`
`SimMode/`	`SimModeBase.*`, `SimModeWorldBase.*`
`SimHUD/`	`SimHUD.cpp`
`Vehicles/Car/`	`SimModeCar.*`
`Vehicles/Multidomain/`	`SimModeWorldMultidomain.*`
`Vehicles/Multirotor/`	`SimModeWorldMultirotor.*`
`Vehicles/ComputerVision/`	`SimModeComputerVision.*`
`Vehicles/SkidVehicle/`	`SimModeSkidVehicle.*`

4.2.3 Step 3: Clean and Rebuild Environment

a. Navigate to `Plugins/AirSim/` and delete its specific `Binaries` and `Intermediate` folders.

b. Delete the following folders from your project root: `Binaries`, `Intermediate`, `Saved`, `DerivedDataCache`.

c. Delete the existing `.sln` file from your project root.

d. Right-click the `.uproject` file → `Show More Options` → `Generate Visual Studio project files`.

e. Open the new `.sln`. Ensure the configuration is set to `DevelopmentEditor` on `Win64`. Press `F5` to build and launch the editor.

4.3 Implementation Details

4.3.1 Core Class Modifications

To support the new multidomain capabilities, the existing Cosys-AirSim class hierarchy for simulation modes was expanded. Figure 7 illustrates this updated architecture, highlighting the addition of the `ASimModeWorldMultidomain` class and its specific position within the inheritance tree.

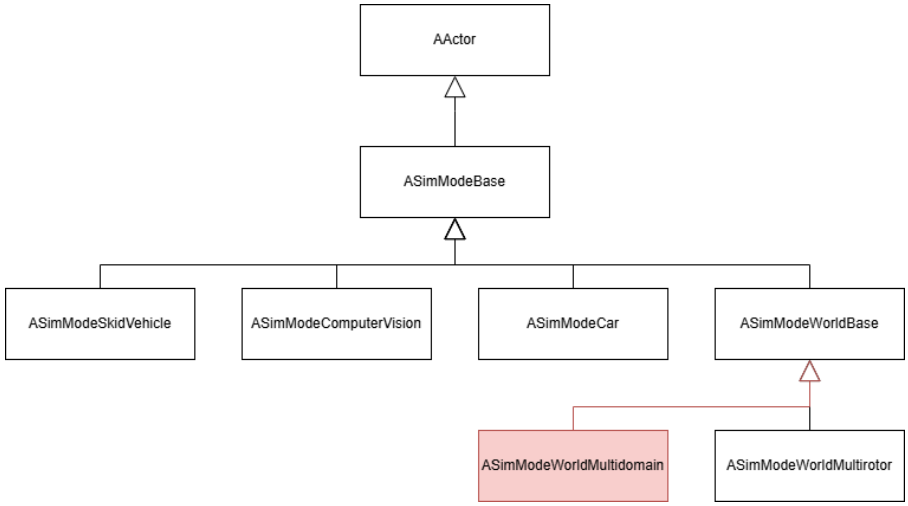

Figure 7: Modified UML Class Hierarchy for Cosys-AirSim Simulation Modes.

Table 1 provides a comprehensive summary of the modifications and additions implemented across the core files to support the multidomain simulation architecture.

Table 1: Summary of Modifications for Multidomain Simulation Modes.

Class	Method / Attribute	Modification Description
`AirSimSettings`	kSimModeTypeMultidomain	Creates a constant, file-specific pointer that points to the text 'Multidomain'.

Class	Method / Attribute	Modification Description
	`loadCameraDirectorSetting`	Defines the camera director follow distance (-8) and Z-position (-3) for Multidomain simulation mode.
	`loadClockSettings`	Sets clock type to `SteppableClock` unless a PX4 enabled vehicle is present (which triggers `ScalableClock`).
	`createDefaultSensorSettings`	Specifies default sensors (IMU, magnetometer, GPS, barometer) for Multidomain mode.
	`loadCoreSimModeSettings`	Enforces "FastPhysicsEngine" as the default physics engine if none is specified in settings.
	`createDefaultVehicle`	Generates a default drone and a default car if Multidomain mode is selected and no vehicles are specified.
`ASimModeBase`	`createApiServer`	Returns a dynamic array of smart pointers.
	`api_servers_` (replacing `api_server_`)	Refactored from a single smart pointer to multiple smart pointers holding multiple objects.
	`pawn_to_vehicle_map`	Added map data structure relating pawn pointers to their vehicle types.
	`addPawnToMap`	Added method to insert entries into the tracking map.
	`getVehicleType`	Added method returning the vehicle type string from the map (or an empty string if not found).
`ASimModeWorldBase`	`initializeForPlay`	Established a bifurcated physics pipeline: drones use AirSim's `PhysicsWorld`, while cars use Unreal Engine's native physics.
	`registerPhysicsBody`	Restricted custom physics registration to dynamically spawned drones, preserving the bifurcated architecture.
	`Tick`	Separated state updates by physics domain to prevent thread conflicts between custom and native physics.
	`reset`	Implemented independent resets for the custom physics world (drones) and Unreal Engine's native physics (cars).
`ASimModeSkidVehicle`	`createApiServer`	Modified to return a dynamic array of smart pointers instead of a single smart pointer.

Class	Method / Attribute	Modification Description
ASimModeWorldMultirotor	createApiServer	Modified to return a dynamic array of smart pointers instead of a single smart pointer.
ASimModeComputerVision	createApiServer	Modified to return a dynamic array of smart pointers instead of a single smart pointer.
	isPaused & pause	Overridden to use Unreal's UGameplayStatics to safely check, stop, or resume simulation execution.
ASimModeCar	createApiServer	Modified to return a dynamic array of smart pointers instead of a single smart pointer.
	isPaused & pause	Overridden to use an internal variable (current_clockspeed_) to pause, or resume by scaling time in Unreal.
ASimModeWorldMultidomain (New Class)	BeginPlay & EndPlay	Initializes the physics world, and safely stops the physics engine before Unreal dismantles the world.
	setupClockSpeed	Configures internal timing, dynamically scaling the physics control loop to allow time manipulation safely.
	createApiServer	Opens two separate RPC servers simultaneously: port 41451 (Multirotors) and port 41452 (Cars).
	getExistingVehiclePawns	Scans environment for drones and cars, pushing them to the internal tracking map with type flags.
	isVehicleTypeSupported	Whitelists kVehicleTypeSimpleFlight and kVehicleTypePhysXCar.
	getVehiclePawnPathName	Acts as a fallback for spawning default blueprints.
	getVehiclePawnEvents getVehiclePawnCameras	Retrieves specific event handlers and camera attachments for a given pawn.
	initializeVehiclePawn createVehicleSimApi	Triggers domain-specific initializations and instantiates core simulation APIs based on the vehicle domain.
ASimHUD	getSimModeFromUser	Added a message to the initial sequence dialogue prompting the user to choose a multidomain or monodomain simulation.

Class	Method / Attribute	Modification Description
	`createSimMode`	Spawns the `ASimModeWorldMultidomain` actor based on user choice or settings.

4.3.2 Development Environment & Toolchain

The development and execution of the simulation environment for the drone and car multi-agent system relied on several key software tools and libraries. Below is the list of the primary tools utilized:

- **Technologies & Software:** Visual Studio Community 2022, Git, Github, LaTeX
- **Platforms:** Desktop Platforms
- **Programming Languages:** C++, Python
- **Frameworks:** Unreal Engine 5.5.4, Cosys-AirSim v3.3
- **Database:** None required
- **Cloud:** None required

The following section details the specific roles and implementation of each tool within the project workflow:

1. **Cosys-AirSim (v3.3.0)**: An advanced, open-source simulator for autonomous vehicles, developed as an extension of Microsoft AirSim by the Cosys-Lab at the University of Antwerp. It is specifically designed for research-focused simulations, offering enhanced support for sensors and multi-agent systems within Unreal Engine 5.5.
2. **Unreal Engine 5.5.4**: A high-performance 3D creation tool used to build the simulation environment. It provides the physics engine (Chaos Physics), photorealistic rendering, and the overall framework for vehicle interaction and environmental dynamics.
3. **Python**: The primary programming language used to develop the control scripts.
4. **C++**: The core programming language used by Unreal Engine and for the development of the Cosys-AirSim plugin.
5. **cosysairsim package**: The official Python client wrapper for the simulation. It provides a Python interface to AirLib, facilitating communication between external control scripts and the Unreal Engine environment via RPC (Remote Procedure Call).
6. **Visual Studio Community 2022**: The Integrated Development Environment (IDE) used for building the C++ components of the plugin and managing the Unreal Engine project dependencies.
7. **LaTeX**: The document preparation system used for the professional typesetting of this project report.

4.4 Scope and Limitations

While our implementation successfully enables the concurrent simulation of multiple vehicle domains, certain technical limitations remain due to the project's development timeline. These constraints are not viewed as failures but rather as opportunities for future work to further enhance the framework:

- **Dynamic Camera Switching:** A significant limitation is the inability to use any built-in mechanism to dynamically switch the main camera view between different vehicles in the Unreal Engine scene during a live simulation. While it is possible to bind the three available Picture-in-Picture (PiP) subwindows to vehicles in `settings.json` before starting, this is strictly limited to three additional vehicles. Furthermore, these windows are quite small, which restricts the user's visibility and makes it difficult to monitor individual agents if they move far apart or encounter unexpected issues.
- **Expanded Vehicle Support:** Currently, supported vehicle types are limited to `kVehicleTypePhysXCar` and `kVehicleTypeSimpleFlight`. Expanding operational support to include additional domains and vehicle types, such as `kVehicleTypeArduRover`, would represent a significant enhancement.

5 Results

The implementation was successfully validated using multiple demonstration scripts. The primary objective of these tests was to ascertain whether the framework could seamlessly spawn, initialize, and control both drones and cars simultaneously within the exact same Unreal Engine 5 instance, without triggering the core engine conflicts that plagued the baseline single-domain Cosys-AirSim.

The test `multi_agent_drone_car` instantiates two independent client objects from the Cosys-AirSim Python API. The extension allows these clients to bind to dedicated Localhost API server ports: port 41451 is exclusively assigned to handle the multirotor control signals, while port 41452 was bound to the car commands.
During the active execution phase, both vehicles achieve real-time, asynchronous operation. The script commands `Drone1` to arm, execute an automated takeoff sequence (`takeoffAsync`), and perform trajectory tracking via `moveToPositionAsync`. In parallel, `Car1` is simultaneously supplied with continuous `CarControls` inputs (throttle set to 0.5, steering set to 0.5), initiating a curved forward movement.

The test `car_hit_drone` specifically validates the physical interaction and collision detection between heterogeneous agents. In this scenario, `Drone1` is placed in a stationary state on the ground, acting as a passive obstacle. `Car1` is then commanded to move forward at full throttle directly towards the UAV. The results demonstrate that the Unreal Engine physics engine, as extended by the framework, correctly processes the collision: the car physically impacts the drone and reflects off it, confirming that the separate API domains do not isolate the agents from the global physics environment.

The test `cars_intersection_crash` evaluates multi-vehicle interaction within the ground domain. Three cars are instantiated: two navigating along a horizontal axis and one along a vertical axis, creating a four-way intersection scenario. The script commands all vehicles to accelerate simultaneously towards the center. Upon impact, the framework accurately simulates the transfer of momentum and torque, resulting in realistic physical behaviors such as vehicle spinning and lateral displacement post-collision, further validating the robustness of the multi-agent physics integration.

The scenario `drone_hit_car` provides a dual-domain collision test where the UAV acts as the kinetic agent. The script initializes `Car1` in a braking state and calculates its vertical position to allow `Drone1` to lock its altitude precisely to the car's chassis level. Upon impact, the framework demonstrates consistent collision geometry: the drone is physically repelled by the car's collider, reflecting the impact. While the stationary car's displacement is negligible due to the significant mass differential and simulated tire friction, the test confirms that the UAV's flight controller and the global physics engine successfully register the collision event across the heterogeneous multi-agent setup.

The test `drones_rapid_cross` focuses on high-velocity multi-agent coordination within the aerial domain. Three UAVs are programmed to perform a synchronized, high-speed crossing maneuver at the same altitude.

During the execution, the system maintains stable control over all three drones until a collision occurs between two of them. The results show that the framework correctly detects the high-speed impact; the involved drones become physically interlocked, demonstrating that the physics engine accurately handles complex mesh-to-mesh interactions even at significant relative velocities.

The `full_chaos` test serves as a stress and stability benchmark for the multi-port API extension. This scenario instantiates a large number of heterogeneous agents, both cars and drones, and commands them to move in divergent, semi-random directions. The simulation results confirm that the framework can simultaneously handle multiple physics-driven interactions: some vehicles successfully navigate open spaces while others collide with static environment geometry (e.g., buildings, walls). Throughout this high-load phase, the communication between the distributed API servers (multirotor and car) remains synchronized, with no observed latency spikes or command drops, validating the scalability of the proposed extension.

The `many_vehicles` scenario validates the system's ability to manage a coordinated fleet of heterogeneous agents. In this test, the framework controls two UAVs and two UGVs simultaneously, while other inert vehicles are present in the environment map. The script executes a synchronized escort pattern where both drones and cars advance in a forward trajectory. The telemetry logs demonstrate that the framework maintains independent and precise control over each specific vehicle instance (e.g., `Drone1`, `Drone2`, `Car1`, `Car2`) across their respective API ports, ensuring that the presence of multiple active agents does not lead to command crosstalk or resource exhaustion within the Unreal Engine instance.

The final test, `swarm_takeoff`, examines the synchronization of multi-agent aerial maneuvers. In this scenario, three drones are commanded to perform a simultaneous takeoff, followed by a rapid vertical ascent to an altitude of 15 meters. The results confirm that the framework can broadcast and synchronize identical high-level commands across multiple UAV instances without jitter or significant timing offset, validating the system's capability for swarm-based research.

The results are definitively positive. Polling the `getCarState` and `getMultirotorState` functions confirm that the vehicles achieve stable initialization and control. Both the UAV and UGV successfully coexist and navigate their distinct physical domains, thereby thoroughly fulfilling the primary objective of the extension. While the framework demonstrates high functional reliability across all tested scenarios, future work will focus on optimizing long-duration stability under complex collision environments to ensure maximum simulation uptime.

6 Conclusion

The development of the multi-domain extension for Cosys-AirSim marks a significant milestone in expanding the simulation capabilities of the Unreal Engine 5 framework. By successfully modifying the core architecture to accommodate different vehicle classes dynamically, this project bridged the gap between single-domain operations and heterogeneous multi-vehicle environments. The validation tests clearly demonstrate the stability of assigning isolated communication interfaces natively, allowing both unmanned aerial vehicles (UAVs) and unmanned ground vehicles (UGVs) to operate concurrently within the same ecosystem.
This enhancement provides researchers, engineers, and developers with a vastly expanded, comprehensive testing ground. The ability to natively simulate cooperative operations unlocks entirely new avenues for testing advanced Artificial Intelligence algorithms, multi-agent path planning, and complex reinforcement learning models in a risk-free, high-fidelity virtual world. Ultimately, this work establishes a robust and critically needed foundation for next-generation multi-agent robotics simulations, empowering the scientific community to innovate safely and effectively before real-world deployment.

References

[1] ANSA. Crolli alla torre dei conti a roma, morto l'operaio rimasto sotto le macerie, 2025.

[2] Alvaro Cantieri, Matheus Ferraz, Guido Szekir, Marco Antônio Teixeira, José Lima, André Schneider Oliveira, and Marco Aurélio Wehrmeister. Cooperative uav–ugv autonomous power pylon inspection: An investigation of cooperative outdoor vehicle positioning architecture. *Sensors*, 20(21), 2020.

[3] Y. Uny Cao, Alex S. Fukunaga, and Andrew B. Kahng. Cooperative mobile robotics: Antecedents and directions. *Autonomous Robots*, 4(1):7–27, 1997.

[4] Chun-Hsian Huang, Yu-Chen Chen, Cheng-Yi Hsu, Jen-Yu Yang, and Chia-Hua Chang. Fpga-based uav and ugv for search and rescue applications: A case study. *Computers and Electrical Engineering*, 119:109491, 2024.

[5] Wouter Jansen, Erik Verreycken, Anthony Schenck, Jean-Edouard Blanquart, Connor Verhulst, Nico Huebel, and Jan Steckel. Cosys-AirSim: A real-time simulation framework expanded for complex industrial applications. In *Proceedings of the 2023 Annual Modeling and Simulation Conference (ANNSIM)*, 2023.

[6] Isuru Munasinghe, Asanka G. Perera, and Ravinesh C. Deo. A comprehensive review of uav-ugv collaboration: Advancements and challenges. *Preprint*, 2024. Available on ResearchGate, CC BY-ND 4.0 License.

[7] Mykola Nikolaiev and Mykhailo Novotarskyi. Comparative review of drone simulators. *ResearchGate*, 2024.

[8] Shital Shah, Debadeepta Dey, Chris Lovett, and Ashish Kapoor. Airsim: High-fidelity visual and physical simulation for autonomous vehicles. In *Field and Service Robotics*, 2017. Microsoft Research, available at https://github.com/microsoft/AirSim.

[9] Farah Syazwani Shahar, Mohamed Tariq Hameed Sultan, Marek Nowakowski, and Andrzej Łukaszewicz. Ugv-uav integration advancements for coordinated missions: A review. *Journal of Intelligent & Robotic Systems*, 111(69):1–15, 2025. Open access, published 02 June 2025.

[10] G. Zhang, S. Wang, Q. Wu, R. Zhang, and K. B. Letaief. Goal-oriented semantic communication for integrated sensing and communication-enabled robotic obstacle avoidance. *IEEE Communications Letters*, 27(12):3255–3259, 2023.